

Week 8 — The Supply Chain (Part 3): Autonomy and Analysis

Where You Are

Two weeks in:

- Week 6: you built the provider and wired the manufacturer to it.
- Week 7: you built the retailer, built the turn engine, and got Claude Code to play **one** role with a skill file.

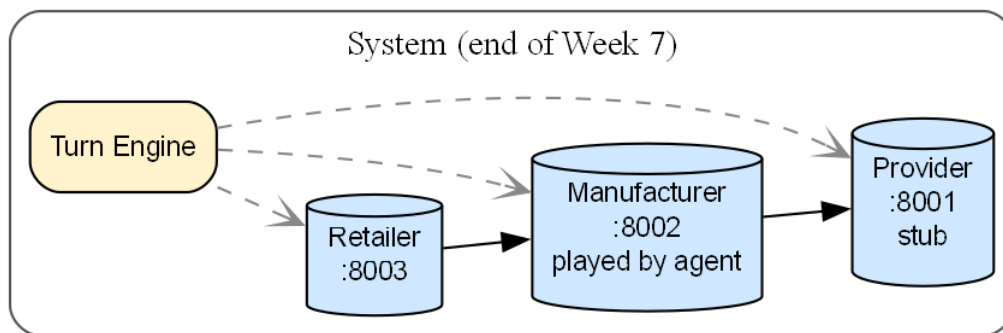


Figure 1: Diagrama

You have **plumbing**. You have **one intelligence**. This week you bring the system to life: every role autonomous, the market pushing back, and enough history in your databases to learn something real.

What You Complete This Week

1. **The remaining two skill files:** `provider-manager.md` and `retail-manager.md`
2. **A real scenario file:** multiple phased market events, not just a smoke test
3. **A full autonomous simulation:** 15–25 days, three agents, market signals, logged end-to-end
4. **Analysis:** charts, comparisons, and a written interpretation of what happened
5. **Presentation:** the system, a live demo, your findings

By the end of today's session:

- All three skills exist and have been tested at least once each
- One full 15+ day simulation has been run against at least one non-trivial scenario
- Charts exist showing inventory, prices, and order fulfillment over time

Finishing polish (second scenario, final report, rehearsed demo) is on your own time before the final deadline.

Part 1: Core Concepts

Multi-Agent Coordination

You now have three independent agents acting simultaneously. They share no memory. They never talk to each other directly. The only thing they share is the state of the world — encoded in the three databases.

This is the shape of real autonomous systems. Uber's surge pricing, Amazon's warehouse robots, an MMO economy — all built from many narrow agents acting on a shared world. They do not need to coordinate explicitly because the world itself coordinates them. A shortage shows up in one agent's stock → it raises a price → the next agent sees the new price → it changes its behaviour.

Your job as a designer is not to **tell** the agents what to do together. It is to give each one a clear local role and let coordination emerge through the world state.

Emergent Behaviour

“Emergence” is the word we use when a system of simple parts produces complex behaviour that no single part was programmed for.

Examples you might see this week:

- **Bullwhip effect:** a small demand spike at the retailer causes a bigger order to the manufacturer, which causes a much bigger order to the provider. Real phenomenon. Named in the 1960s. You will probably reproduce it accidentally.
- **Price wars / collapses:** if retailers are too aggressive lowering prices during low demand, they race each other to the floor.
- **Stockout cascades:** one agent’s slow reaction to a signal blocks everyone downstream.

These are features, not bugs. Emergence is what makes this project interesting. When you see an unexpected behaviour, your job is to **explain it**, not to hide it.

Observability

Three agents making decisions produce a lot of output. Without discipline, you will have no idea what happened.

What you need:

1. **Per-turn agent logs** (already captured from Week 7). One file per role per day.
2. **Per-event log in each database.** The `events` table. Query it with SQL for analysis.
3. **Numeric time-series.** Stock levels, prices, fulfillment rates — sampled once per simulated day. A dedicated `metrics` table or a snapshot-on-advance routine works.
4. **A summary line per turn.** After each day, the engine prints a one-line summary: `Day 7: 12 customer orders / 9 fulfilled / 2 backordered / 1 stockout.`

Without these, your analysis in Part 5 will be impossible.

Scenario Design

A good scenario is not just “demand changes.” A good scenario puts agents under **specific pressure** and lets you see whether they handle it.

Patterns that produce interesting runs:

- **The slow ramp:** demand builds over 5–7 days. Do agents build stock in time?
- **The sudden shock:** demand triples overnight (Black Friday). Do agents respond in time, or do they stock out?
- **The combined stress:** demand spikes *and* suppliers slow down at the same time (chip shortage during Christmas). This is when the bullwhip shows up.
- **The quiet period:** demand drops. Do agents correctly lower prices and drain inventory, or do they panic-buy?

You will run at least two scenarios and compare them.

Part 2: Complete the Skill Set

Skill: Provider Manager

Create `skills/provider-manager.md`:

```
# Skill: Provider Manager
```

```
## Your Role
```

You manage a parts supply company. Each simulated day:

1. Process incoming purchase orders from manufacturers
2. Manage your stock (simulated upstream supply)
3. Adjust prices based on stock pressure
4. Ship orders whose lead time has elapsed

Available Commands

Check current state

- `./provider-cli day current``
- `./provider-cli stock``
- `./provider-cli orders list`` (optional: `--status pending``)
- `./provider-cli orders show <id>``

Operations

- `./provider-cli restock <product> <quantity>``
- `./provider-cli price set <product> <tier> <price>``

DO NOT

- Do NOT call `day advance``. The turn engine does that.
- Do NOT change a tier's price more than 15% in one day.
- Do NOT let any single product go to zero stock if orders for it are pending.

Decision Framework

1. ****Assess.**** Run `stock`` and `orders list``. Summarise the state in 2-3 sentences.
2. ****Restock.**** If any product stock is below 50% of its starting level, restock up to the starting level. Log the rationale.
3. ****Adjust prices.**** If stock of a product is above 150% of starting, lower the top tier price 5-10%. If stock is below 30%, raise it 5-10%. Stay within the 15% daily bound.
4. ****Summarise.**** 3-5 bullet points of what you did today and why.

Market Signals

- `supply_modifier < 0.7``: shortage context. Raise prices more aggressively; accept that you may not be able to fulfill all orders.
- `demand_modifier > 1.5``: manufacturer will likely place larger orders. Build stock ahead.

Skill: Retail Manager

Create skills/retail-manager.md:

Skill: Retail Manager

Your Role

You manage a retail store that sells 3D printers to end customers. Each simulated day:

1. Fulfill customer orders from stock where possible
2. Mark insufficient-stock orders as backordered
3. Order more printers from the manufacturer if stock is low
4. Set retail prices to balance profit against demand

Available Commands

Check current state

- `./retailer-cli day current``

```

- `./retailer-cli stock`
- `./retailer-cli customers orders`
- `./retailer-cli customers order <id>`

### Fulfillment
- `./retailer-cli fulfill <order_id>`
- `./retailer-cli backorder <order_id>`

### Purchasing
- `./retailer-cli purchase list`
- `./retailer-cli purchase create <model> <qty>`

### Pricing
- `./retailer-cli price list`
- `./retailer-cli price set <model> <price>`

## DO NOT
- Do NOT call `day advance`. The turn engine does that.
- Do NOT set retail price below manufacturer wholesale + 20%.
- Do NOT leave customer orders in `pending`. Every one becomes `fulfilled` or `backordered` by
  ↪ end of turn.

## Decision Framework

1. **Fulfill.** For each pending customer order, fulfill if stock exists, otherwise backorder.
2. **Reorder.** For each model where stock is below 3 days of recent average demand, place a
  ↪ purchase order with the manufacturer.
3. **Price.** If stock is low relative to recent demand, raise price 5%. If stock is piling up
  ↪ (over 5 days supply) and prices are not already at floor, lower price 5%.
4. **Summarise.** Orders fulfilled, backordered, purchases placed, price changes - one line
  ↪ each.

## Market Signals

- `demand_modifier > 1.5`: demand spike incoming. Place larger purchase orders now; prices may
  ↪ still hold.
- `demand_modifier < 0.8`: soft demand. Slow reorders; consider cutting prices.
- `price_sensitivity: high`: customers are shopping around. Be cautious about raising prices.

```

Testing Each Skill Individually

Before running all three agents together, test each one in isolation. Set the other two roles back to the stub in the turn engine. Run one day. Watch what the agent does. If the skill is clear, the behaviour is predictable.

Only once all three work in isolation should you run them together.

Part 3: Scenario Files — The Real Thing

```

{
  "scenario_name": "Q4 2026 - Holiday Rush with Chip Shortage",
  "base_demand": {"mean": 5, "variance": 2},
  "base_price": 400,
  "events": [
    {
      "name": "normal",

```

```

    "start_day": 1,
    "end_day": 10,
    "demand_modifier": 1.0,
    "supply_modifier": 1.0,
    "description": "Business as usual"
  },
  {
    "name": "black_friday",
    "start_day": 11,
    "end_day": 13,
    "demand_modifier": 3.0,
    "supply_modifier": 1.0,
    "price_sensitivity": "high",
    "description": "Black Friday weekend - demand triples, customers price-sensitive"
  },
  {
    "name": "chip_shortage",
    "start_day": 14,
    "end_day": 20,
    "demand_modifier": 1.5,
    "supply_modifier": 0.4,
    "lead_time_modifier": 2.0,
    "description": "Chip shortage - providers have less stock and longer lead times"
  },
  {
    "name": "christmas_season",
    "start_day": 18,
    "end_day": 25,
    "demand_modifier": 2.5,
    "supply_modifier": 0.6,
    "description": "Christmas rush during constrained supply"
  }
]
}

```

Notes:

- Overlapping events (days 18–20 have both chip shortage and Christmas) should compound. Decide how in your engine — multiply modifiers, or take the max — and document the choice.
- `lead_time_modifier` affects the provider's lead times (multiply the product's base lead time by this). Apply it during provider `day_advance` logic.
- `price_sensitivity` is a hint to agents via the signal, not a hard engine rule.

Create at least **two** scenario files:

1. `scenarios/calm-market.json` — steady baseline; demand stable; no disruptions. Your control group.
2. `scenarios/holiday-rush.json` — the one above, or your own volatile scenario with at least two overlapping events.

Part 4: Running Extended Simulations

One Full Run

```
python turn_engine.py config/sim.json scenarios/holiday-rush.json 25
```

25 simulated days. Three agents active every turn. Demand fluctuating. Logs accumulating.

Time-box this: one run should not take more than ~30 minutes of wall clock. If it takes longer, your per-turn agent prompts are too big or your timeout is too permissive. Fix that first.

Metrics to Capture

During `day advance`, each app should snapshot these into a `metrics` table (or equivalent):

App	Metrics
Provider	stock per product, current price per product/tier, orders pending/shipped/delivered today
Manufacturer	parts stock, finished-printer stock, production utilisation, wholesale price per model, sales orders pending/completed
Retailer	printer stock per model, retail price per model, customer orders placed/fulfilled/backordered

Add a `sim_day` column and query by day. This is what you will plot.

What Can Go Wrong

Expect:

- An agent making a bad call and cascading into a crisis. Do not rewind — watch what happens.
- A skill ambiguity revealing itself mid-run. Note it, let the run finish, fix the skill for the next run.
- Stuck agents (timeouts). Log them, move on. A system that survives one flaky agent is more interesting than a perfect one.

The goal is not a clean run. The goal is a **readable** run.

Part 5: Analysis

Required Charts

For each scenario run, produce at least:

1. **Inventory over time** — one chart, three lines: parts stock at manufacturer, finished-printer stock at manufacturer, printer stock at retailer
2. **Prices over time** — one chart, three lines: provider price (one representative part), manufacturer wholesale, retailer retail
3. **Order fulfillment** — daily bar chart: for each day, customer orders placed vs fulfilled vs backordered
4. **Events overlay** — a strip chart marking when each scenario event started/ended, aligned with the above

Use matplotlib. Embed the charts in your report.

Required Interpretation

For each chart, write 2–4 sentences explaining what you see. Not “the line goes up.” Explain the causal chain: *why* the line goes up on day 12, what agent decision produced it, whether the signal did what you expected.

Specifically answer these questions about your volatile-market run:

- Did the manufacturer build stock ahead of Black Friday? If yes, how? If no, why not?
- When stockouts happened, whose decision was the proximate cause? Whose was the root cause?
- Did prices stabilise or oscillate? If they oscillated, what drove the oscillation?
- Can you identify a bullwhip moment — a case where demand variance amplified upstream?

Scenario Comparison

Plot equivalent metrics for the calm and volatile scenarios side-by-side. Write a paragraph comparing them: what do the agents do in one they do not do in the other, and is that a success or a failure?

Part 6: Verification Checklist

Before the final demo:

- ☐ All three skill files exist and have each been tested in isolation
 - ☐ Full turn with all three agents runs clean for at least one day
 - ☐ A 15+ day simulation has completed end-to-end against a non-trivial scenario
 - ☐ At least two scenario files exist and have been run
 - ☐ Charts exist for inventory, prices, and fulfillment
 - ☐ Event logs in all three databases are non-empty and coherent
 - ☐ Agent per-turn logs are saved to `logs/` for post-hoc review
 - ☐ Presentation slides are drafted
 - ☐ Demo scenario chosen and rehearsed (short — ~3 days for a live run)
-

Part 7: Final Deliverables

This is the end of the three-week arc. Everything below is part of the final submission.

1. GitHub Repository

- Source for all three apps (`provider/`, `manufacturer/`, `retailer/`)
- Turn engine (`turn_engine.py` or `engine/`)
- All skill files (`skills/provider-manager.md`, `skills/manufacturer-manager.md`, `skills/retail-manager.md`)
- At least two scenario files (`scenarios/`)
- Seed data for every app
- Final `CLAUDE.md`
- Final `docs/PRD.md` — now reflecting the full system
- Final `README.md` with reproducible setup + run instructions
- `.gitignore` excluding `.env`, `__pycache__/`, `.venv/`, `*.db`, `logs/`
- Clean commit history with issue references across all three weeks

2. Final Report (5–8 pages, PDF)

Generated via pandoc + mermaid-filter.

Structure:

- a) System architecture** - Full system diagram - Data model for each app (one ER diagram per app is fine) - Turn engine design: order of operations, why - How market signals flow through the system
- b) Agent design** - Your three skill files (summarise, do not paste in full if you can help it) - Decisions you made during skill authoring — and skills you rewrote after watching the agent fail - What the agents are good at and what they are bad at
- c) Simulation results** - Charts from at least one 15+ day run of the volatile scenario - Charts from the calm scenario for comparison - Your interpretation — causal chains, not just observations - Any emergent behaviour you noticed, with an explanation
- d) Vibe-coding reflection** - How you used Claude Code across all three weeks - What worked. What did not. - The one thing you would redesign if you started over

3. Presentation (max 10 slides + live demo)

Slides should cover:

- System overview (one architecture diagram slide)
- Agent design (one slide per role, brief)
- Live demo: run 2–3 days of simulation, watch the output, narrate what the agents are doing
- Results: one “most interesting chart” slide
- Reflection: one slide on what you learned

The demo is the most important part. Rehearse it. Keep it short. Have a plan for what to do if the agents stall.

Write It Yourself

Same rule as every week. Use LLMs to build the software. Use Claude Code to play the agent roles. Write the report in your own words. If your report reads like it could have been written by someone who never watched a run, it is not good enough.

Part 8: Stretch Goals

Only after the required deliverables work. In rough order of value:

1. **LLM-driven end customers.** Replace the deterministic demand generator with an LLM skill for a customer persona. Adds cost; adds realism.
2. **Multiple retailers competing.** Run two retailers with different pricing strategies and see which one captures more demand.
3. **A fourth event type: supply disruption by region.** Half of your providers go down for 3 days. How does the manufacturer reroute?
4. **A “shock test” report:** write a one-paragraph scenario description in plain English, have an LLM generate the scenario JSON, run it, and see what happens. This tests how generalisable your system is.
5. **Dashboard.** Streamlit app that tails the metrics tables and shows live charts during a run. Visually satisfying; useful during the demo.

None of these are required. All of them are good.

API Key Safety

Same rules. Last week.

```
.env
__pycache__/
.venv/
*.db
logs/
```

Summary

Week	What was built	End state
6	Provider app + manufacturer wired to it	Two apps talking over REST
7	Retailer + turn engine + first skill	One agent making real decisions
8	Two more skills + scenarios + full run + analysis	Autonomous supply chain, analysed

You now have:

- **Three services** with their own state and their own interfaces
- **Three agents** with clear roles, each a markdown file
- **An engine** that orchestrates them through time
- **Scenarios** that inject controlled pressure
- **Data** that lets you reason about what happened

This architecture is not a toy. It is the shape of real autonomous business systems: deterministic software handles execution, LLMs handle strategy, well-defined interfaces keep everything auditable and under control.

The hardest part of the three weeks was not any single component. It was keeping your head across all of them — deciding when to trust the agent, when to rewrite the skill, when to fix the plumbing, when to change the scenario. That judgment is what this project was really teaching you.

Go demo it well.